

GuardianIQ

Phase 2 Developer Specification

Governed Workflow Scheduling & AI Agent Execution

End-to-End Specification covering Frontend, Backend, Data Model, APIs, Governance Controls, Audit, Testing and Delivery Readiness

Field	Value
Platform	GuardianIQ
Phase	Phase 2
Phase Name	Governed Workflow Scheduling & AI Agent Execution
Prepared For	GuardianIQ Product, Architecture and Development Teams
Prepared By	Innovant.ai Architecture & Product Team
Version	v1.0
Primary Input	GuardianIQ Domain 1 Foundation: Identity, Access & Accountability
Pre-requisite	Phase 0 foundation and Phase 1 Governance Registry completed

Core design statement

GuardianIQ Phase 2 must allow AI agents and AI models to be scheduled, executed and monitored only through governed workflows where ownership, permission boundaries, policy checks, audit events, run history, outputs, failures and human oversight are traceable end to end.

Table of Contents

1. Executive Summary
2. Phase 2 Scope and Boundaries
3. Phase 2 Functional Requirements
4. Reference Alignment and Governance Principles
5. Target Operating Flow
6. User Personas and Roles
7. End-to-End Functional Workflows
8. Frontend Specification
9. Backend Architecture Specification
10. Data Model and PostgreSQL Physical Design
11. API Contract Specification
12. Scheduling and Execution Engine Design
13. RBAC + ABAC Authorization Design
14. Governance Event and Audit Design
15. Notifications, Escalations and SLA Handling
16. Error Handling and Failure Recovery
17. Security, Privacy and Compliance Controls
18. Integration Design
19. Development Work Breakdown Structure
20. Testing and Acceptance Criteria
21. Deployment and Migration Guide
22. Seed Data and Sample Scenarios
23. Implementation Risks and Controls
24. Appendices: DDL, Payloads and Developer Checklists

1. Executive Summary

Phase 1 completed the GuardianIQ registry foundation for AI models, AI agents, tools, workflows, applications and data sources. Phase 2 turns these registry assets into governed, schedulable execution flows. The purpose is to let GuardianIQ schedule AI agents and models to perform controlled governance activities, produce findings, generate recommendations, and log complete traceability without allowing uncontrolled autonomous execution.

This phase is intentionally designed as a governed workflow execution capability, not as a simple scheduler or cron wrapper. Every schedule, run, agent invocation, policy check, result, failure and override must be controlled by identity, permissions, ownership, ABAC context, audit events and governance status.

Phase 2 Capability	Meaning for GuardianIQ	Why it matters
Workflow scheduling	Configure recurring, one-time, manual and event-triggered governed workflows.	Enables repeatable governance checks and operational AI oversight.
AI agent assignment	Attach one or more registered agents/models to a workflow schedule.	Uses Phase 1 registry assets in governed execution.
Execution boundaries	Restrict allowed tools, data sources, operations, confidence thresholds and output types.	Prevents AI agents from acting outside approved scope.
Run tracking	Persist each scheduled run, step, status, output, exception and timing.	Provides operational visibility and traceability.
Human oversight	Route activation, risky outputs or failed runs to reviewers or approval groups.	Aligns with governance-by-responsibility.
Audit-first design	Generate immutable governance events at every critical step.	Supports compliance, investigation and accountability.

1.1 Source Document Interpretation

The attached Domain 1 foundation document defines GuardianIQ as an accountability-first platform where governance is meaningful only when responsibility is assigned and traceable. It establishes hybrid RBAC + ABAC, machine actors, delegation, approval groups, and immutable governance events as core requirements. Phase 2 uses those identity concepts as mandatory controls for workflow scheduling and agent execution.

Domain 1 Concept	How Phase 2 Uses It
User, role, permission	Controls who can create schedules, activate workflows, view run output, approve high-risk activation and manage failures.
Machine actor	AI agents are treated as governed actors with owner, execution mode, allowed tools and auditability.
Approval group	Used to approve schedule activation, high-risk execution modes, critical workflow failures and escalation routing.
Delegation	Allows temporary delegated activation/review when assigned owners or approvers are unavailable.
Responsibility mapping	Every workflow schedule, agent assignment and run output must have owner/reviewer/escalation owner where applicable.
Governance events	Every important identity, access, scheduling, execution and override action generates audit events.

2. Phase 2 Scope and Boundaries

2.1 In Scope

- Create, update, activate, pause, resume and retire governed workflow schedules.
- Assign registered AI agents and AI models to workflow schedules.
- Configure schedule type: manual, one-time, hourly, daily, weekly, monthly and cron expression.
- Configure execution boundaries: allowed tools, allowed data sources, blocked operations, max runtime, retry rules and confidence threshold.
- Run scheduled workflows and persist workflow run history, run steps, outputs, findings, recommendations and failures.
- Apply RBAC + ABAC authorization checks before schedule creation, activation, execution and output visibility.

- Generate governance events for schedule lifecycle, execution lifecycle, policy checks, failures, denials and escalations.
- Provide frontend screens for scheduler management, agent assignment, run history, run detail, audit timeline and authorization simulator.
- Provide backend APIs and services for scheduling, run orchestration, permission checks, event logging and run monitoring.
- Extend PostgreSQL data model for scheduling, execution, run output, failure handling, authorization decisions and notifications.

2.2 Out of Scope for Phase 2

- Full Phase 3 policy rule engine with complex nested policy evaluation. Phase 2 will support lightweight policy hooks and authorization checks.
- Autonomous write execution into enterprise systems. Initial execution mode should be READ_ONLY, RECOMMEND_ONLY or APPROVAL_REQUIRED.
- Advanced AI evaluation metrics, hallucination detection or model drift analytics. These can be added later.
- Full graph database implementation. Relationships continue to be represented using PostgreSQL relational and JSONB structures.
- External customer production integrations unless a controlled demo connector is explicitly approved.

3. Phase 2 Functional Requirements

Requirement ID	Requirement	Priority
P2-REQ-001	The platform shall allow authorized users to create governed workflow schedules linked to a registered workflow.	Must
P2-REQ-002	The platform shall allow assignment of one or more AI agents to a workflow schedule.	Must
P2-REQ-003	Each agent assignment shall define execution mode, model, allowed tools, allowed data sources and boundary rules.	Must
P2-REQ-004	The platform shall support manual, one-time, interval, daily, weekly, monthly and cron-based scheduling.	Must
P2-REQ-005	Every active schedule shall have an owner, status, timezone, activation approver or approval group when required.	Must
P2-REQ-006	High-risk schedules or schedules with execution-capable tools shall require approval before activation.	Must
P2-REQ-007	The platform shall persist each run as workflow_run with run status, trigger type, timestamps, actor and summary.	Must
P2-REQ-008	The platform shall persist run steps, including policy check, agent invocation, output parsing and audit publishing.	Must
P2-REQ-009	The platform shall persist structured outputs including findings, recommendations, evidence and risk summary.	Must
P2-REQ-010	The platform shall support retries and failure handling based on configured retry policy.	Should
P2-REQ-011	The platform shall generate governance events for schedule lifecycle and execution lifecycle events.	Must
P2-REQ-012	The platform shall expose run history, run detail and audit timeline in the frontend.	Must
P2-REQ-013	The platform shall support RBAC + ABAC authorization decision before sensitive actions.	Must
P2-REQ-014	The platform shall support pausing or disabling schedules without deleting historical runs.	Must
P2-REQ-015	The platform shall prevent overlapping runs for schedules where concurrency is disabled.	Must
P2-REQ-016	The platform shall support escalation and notification if a run fails, breaches SLA or generates high-risk findings.	Should
P2-REQ-017	The platform shall support sample/demo workflow templates to accelerate developer and business testing.	Should
P2-REQ-018	The platform shall provide API-level pagination, filtering, sorting and standard error responses.	Must

4. Reference Alignment and Governance Principles

Phase 2 is aligned with accountability-first AI governance, AI management system thinking, risk-based oversight and context-aware authorization. NIST AI RMF frames AI risk management as a structured practice for managing risks to individuals, organizations and society. ISO/IEC 42001 defines requirements and guidance for establishing, maintaining and improving an AI management system. NIST SP 800-162 ABAC defines authorization based on attributes of subjects,

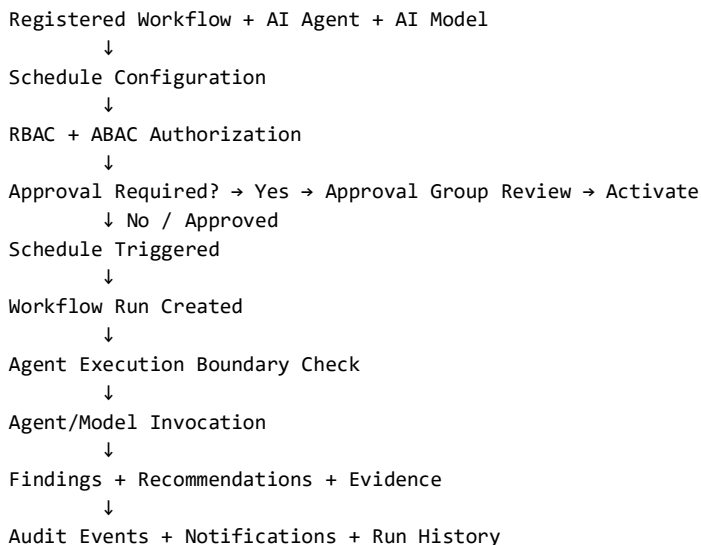
objects, operations and environment. The EU AI Act direction emphasizes human oversight, monitoring and logging for high-risk AI systems. These references support GuardianIQ's decision to implement schedule-based AI activity with human oversight, permissions, logging and accountability.

Principle	Phase 2 Design Implication
Governance-by-responsibility	Every schedule and run must have owner, reviewer or escalation path where needed.
Least privilege	Agent assignments must explicitly list allowed tools and operations.
Human oversight by design	High-risk or execution-capable schedules require approval before activation.
Context-aware authorization	Activation and execution depend on user role plus risk, department, execution mode and tool sensitivity.
Audit-first design	All schedule and run lifecycle changes publish governance events.
Machine actors are governed actors	AI agents are treated as accountable non-human actors linked to owners and permission boundaries.

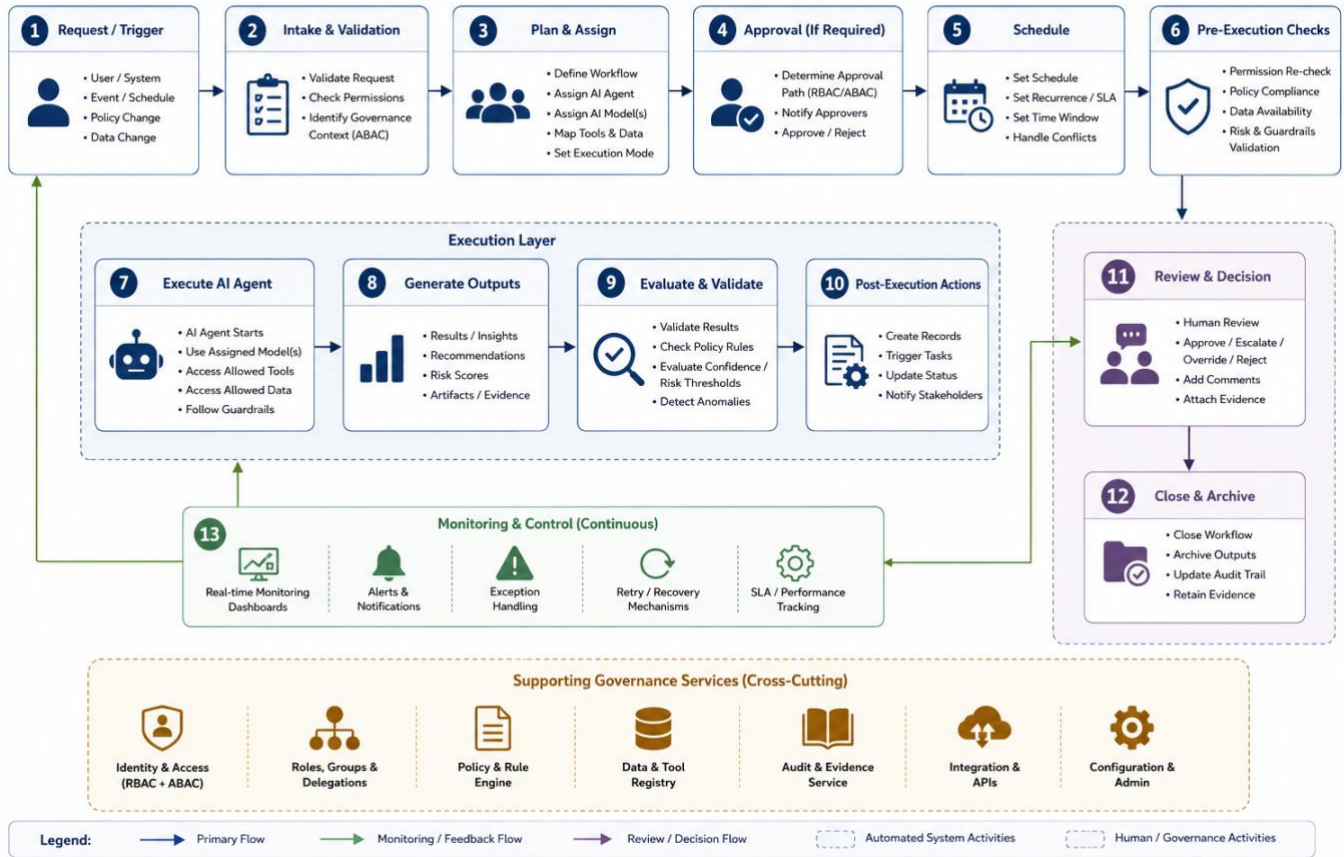
5. Target Operating Flow

1. Authorized user opens Workflow Scheduler and selects a registered workflow from Phase 1.
2. User selects one or more registered AI agents and models from Phase 1 registry.
3. User configures schedule frequency, timezone, start date, execution mode, allowed tools and boundaries.
4. GuardianIQ validates registry relationships, ownership and mandatory fields.
5. GuardianIQ runs RBAC + ABAC authorization check for schedule creation.
6. If schedule is high-risk, write-enabled, or critical, status becomes PENDING_APPROVAL.
7. Approval group reviews and activates schedule. Activation event is logged.
8. Scheduler engine triggers workflow run at configured time or manual trigger.
9. Workflow execution service creates run record and run steps.
10. Agent runtime service invokes AI agent within boundaries and captures outputs.
11. Output parser creates findings, recommendations, risk summary and evidence records.
12. Governance event publisher logs run started/completed/failed and output events.
13. Run history and dashboard update for monitoring and audit.

5.1 High-Level Process Diagram in Text



5.1 High-Level Process Diagram in Text



6. User Personas and Roles

Persona / Role	Primary Phase 2 Actions	Restrictions
SYSTEM_ADMIN	Configure platform scheduler settings, view all runs, manage global seed data.	Should not business-approve high-risk AI output by default.
GOVERNANCE_ADMIN	Create workflow schedules, configure agent assignment, manage schedule lifecycle.	Activation of high-risk schedule may still require approval.
AI_ASSET_OWNER	Review schedules using their model/agent, update boundaries, view related runs.	Can manage only assigned assets unless additional permissions granted.
AI_REVIEWER	Review outputs, findings, generated recommendations and run evidence.	Review/comment only unless approval permission is assigned.
BUSINESS_APPROVER	Approve business workflow schedule activation within allowed department/risk scope.	Cannot approve outside ABAC scope.
RISK_MANAGER	Approve or escalate high-risk schedules and high-risk run findings.	Should be used for critical/high-risk workflows.
COMPLIANCE_OFFICER	Review compliance-sensitive schedules and run evidence.	Read/review/approve based on policy and sensitivity.
AUDITOR	View run history, audit timeline and evidence.	Read-only; no changes.
AI Agent / Machine Actor	Execute assigned workflow step using allowed model/tools.	Must not exceed boundary rules or execution mode.

7. End-to-End Functional Workflows

Workflow	Description	Expected Result
----------	-------------	-----------------

WF-01 Create Schedule	Governance Admin creates a schedule for a registered workflow and assigns an AI agent.	DRAFT schedule created; event <code>WORKFLOW_SCHEDULE_CREATED</code> logged.
WF-02 Activate Schedule	Approver reviews risk, agent boundary and execution mode before activation.	Schedule status <code>ACTIVE</code> ; activation audit event logged.
WF-03 Manual Run	Authorized user manually triggers an active schedule.	Run created with <code>trigger_type</code> <code>MANUAL</code> .
WF-04 Scheduled Run	Scheduler automatically triggers due schedule.	Run created with <code>trigger_type</code> <code>SCHEDULED</code> .
WF-05 Agent Boundary Check	Before agent invocation, system verifies execution mode, allowed tools and authorization.	Execution allowed or blocked with <code>AUTHORIZATION_DENIED</code> event.
WF-06 Run Completion	Run completes and outputs are stored.	Run status <code>COMPLETED</code> ; findings/recommendations visible.
WF-07 Run Failure	Execution fails due to tool error, timeout, model error or authorization denial.	Run status <code>FAILED</code> ; failure record created; notification/escalation triggered.
WF-08 Pause Schedule	Owner or admin pauses schedule temporarily.	Schedule status <code>PAUSED</code> ; no future automatic triggers until resumed.
WF-09 Retire Schedule	Admin retires obsolete schedule.	Schedule status <code>RETIRED</code> ; history preserved.
WF-10 High-Risk Findings	Agent output includes high-risk finding.	Recommendation created or flagged; risk/compliance reviewer notified.

8. Frontend Specification

Frontend should be implemented as a workflow-oriented experience, not as isolated CRUD screens. Users must understand what is scheduled, which AI assets are involved, what permissions are allowed, when it runs, and what governance controls apply.

8.1 Route Map

Route	Screen	Purpose
<code>/workflow-scheduler</code>	Workflow Scheduler Dashboard	List schedules with status, next run, owner, risk and health.
<code>/workflow-scheduler/new</code>	Create Schedule Wizard	Step-by-step creation of governed schedule.
<code>/workflow-scheduler/:id</code>	Schedule Detail	View configuration, agents, runs, events and approvals.
<code>/workflow-runs</code>	Run History	Search/filter all workflow runs.
<code>/workflow-runs/:runId</code>	Run Detail	Full run trace, steps, output, evidence and audit events.
<code>/agent-assignments</code>	Agent Assignment Matrix	Manage workflow-to-agent/model/tool relationships.
<code>/authorization-simulator</code>	Authorization Simulator	Test if actor/agent can perform action in context.
<code>/schedule-approvals</code>	Schedule Approval Queue	Approve/reject schedule activation where required.

8.2 Create Schedule Wizard

Step	Fields / Controls	Validation
Step 1 - Select Workflow	Workflow dropdown, workflow owner, criticality, description.	Workflow must be <code>ACTIVE</code> and not deleted.
Step 2 - Assign AI Agent	Agent dropdown, model dropdown, execution mode, confidence threshold.	Agent and model must be <code>ACTIVE</code> ; execution mode cannot exceed agent registry mode.
Step 3 - Tool & Data Boundaries	Allowed tools, allowed data sources, blocked operations, max records, max runtime.	Write/execute tools trigger approval requirement.
Step 4 - Schedule	Schedule type, cron expression, timezone, start/end date, concurrency rule, retry policy.	Cron valid; start date $\geq$ current date; timezone required.
Step 5 - Governance Controls	Owner, reviewer, approval group, risk level, SLA, notification recipients.	High-risk schedules require approval group.
Step 6 - Review & Submit	Summary, validation results, activation requirement, submit button.	Submit creates <code>DRAFT</code> or <code>PENDING_APPROVAL</code> schedule.

8.3 Component Inventory

Component	Purpose	Key Props
<code>ScheduleListTable</code>	Displays schedules with filters and actions.	<code>schedules</code> , <code>filters</code> , <code>onPause</code> , <code>onResume</code> , <code>onOpen</code>
<code>ScheduleStatusBadge</code>	Shows <code>DRAFT</code> , <code>PENDING_APPROVAL</code> ,	<code>status</code>

	ACTIVE, PAUSED, FAILED, RETIRED.	
AgentAssignmentPanel	Assigns agent/model/tool/data boundaries.	workflowId, agentOptions, modelOptions
CronExpressionBuilder	Builds cron expression with friendly UI.	scheduleType, cronExpression, timezone
BoundaryRuleEditor	Edits allowed operations and restrictions.	allowedTools, blockedOperations, boundaryRules
ApprovalRequirementBanner	Explains why approval is required.	riskLevel, executionMode, toolSensitivity
RunHistoryTable	Shows workflow run history.	runs, filters, onOpenRun
RunTimeline	Step-by-step run trace and audit events.	runSteps, auditEvents
RunOutputViewer	Shows findings, recommendations, evidence and raw output.	outputs, findings, evidence
AuthorizationDecisionPanel	Shows allow/deny result and rule explanation.	decision, reasons, evaluatedAttributes

8.4 Screen-Level Acceptance Criteria

- Schedule Dashboard must load within 2 seconds for 500 schedules using pagination.
- Create Schedule Wizard must not allow activation without required owner and agent assignment.
- High-risk schedules must clearly show approval requirement before submission.
- Run Detail must show run status, timing, steps, output and audit events.
- User must not see schedules/runs outside authorized scope.
- All save/submit actions must show backend validation errors in user-friendly language.

9. Backend Architecture Specification

Backend should follow modular FastAPI service design. The scheduler capability should be isolated into dedicated modules while reusing Phase 0 identity, Phase 1 registry and audit services.

9.1 Recommended Backend Modules

Module	Responsibility	Depends On
workflow_scheduler	Schedule CRUD, status changes, validation, lifecycle events.	core, registry, audit
workflow_execution	Run creation, run step orchestration, state transitions.	workflow_scheduler, registry
agent_runtime	Agent/model invocation adapter, boundary checks, output capture.	registry, authorization
authorization	RBAC + ABAC decision service.	core, registry
event_publisher	Governance event publishing.	audit
notification_service	Email/Teams/in-app notification placeholders.	core, workflow_execution
scheduler_worker	Background worker to detect and execute due schedules.	workflow_scheduler, workflow_execution
run_output_service	Findings, recommendation and evidence persistence.	workflow_execution, audit

9.2 Suggested FastAPI Folder Structure

```

backend/
  app/
    main.py
    api/
      phase2_scheduler_routes.py
      phase2_run_routes.py
      phase2_agent_assignment_routes.py
      phase2_authorization_routes.py
    modules/
      workflow_scheduler/
        models.py
        schemas.py
        repository.py
        service.py
        validators.py
      workflow_execution/
        models.py
        schemas.py

```



```

repository.py
service.py
state_machine.py
agent_runtime/
  service.py
  adapters.py
  boundary_checker.py
  output_parser.py
authorization/
  rbac_service.py
  abac_service.py
  decision_service.py
notifications/
  service.py
audit/
  event_service.py
workers/
  scheduler_worker.py
shared/
  db.py
  errors.py
  pagination.py
  logging.py
  enums.py

```

9.3 Core Backend Services

Service	Public Methods	Important Notes
WorkflowScheduleService	create_schedule, update_schedule, submit_for_approval, activate, pause, resume, retire, get_due_schedules	Must enforce validation and authorization.
WorkflowRunService	create_run, start_run, complete_run, fail_run, add_step, update_step	Responsible for run state machine.
AgentRuntimeService	validate_agent_boundary, invoke_agent, parse_output	No external execution without boundary check.
AuthorizationDecisionService	can_create_schedule, can_activate_schedule, can_run_schedule, can_view_output	Combines RBAC + ABAC + relationship checks.
GovernanceEventService	publish_event, publish_batch, get_timeline	Must create immutable audit events.
ScheduleNotificationService	notify_approval_required, notify_run_failed, notify_high_risk_output	Can start with database/in-app notification table.
ScheduleValidationService	validate_cron, validate_agent_assignment, validate_boundaries, validate_approval_requirement	Returns structured validation result to frontend.

10. Data Model and PostgreSQL Physical Design

Phase 2 extends the Phase 0 and Phase 1 PostgreSQL foundation. The recommended schema for these new tables is workflow. Some cross-cutting records may use audit and governance schemas. The model should preserve phase-wise implementation while supporting future policy and approval modules.

10.1 New Tables Required in Phase 2

Table	Purpose	Key Fields
workflow.workflow_schedules	Master schedule configuration for a workflow.	workflow_id, schedule_type, cron_expression, timezone, owner_user_id, approval_group_id, status
workflow.workflow_schedule_agent_assignments	Agent/model/tool boundary assignment per schedule.	schedule_id, agent_id, model_id, execution_mode, allowed_tools_json, boundary_rules_json
workflow.workflow_runs	Every triggered execution instance.	schedule_id, workflow_id, run_status, trigger_type, started_at, completed_at, summary
workflow.workflow_run_steps	Step-level execution trace.	run_id, step_code, step_status, started_at, completed_at, input_json, output_json

workflow.workflow_run_outputs	Structured outputs generated by agents/models.	run_id, output_type, severity, risk_score, findings_json, recommendations_json, evidence_json
workflow.workflow_run_failures	Failure records and diagnostic details.	run_id, failure_type, failure_message, retry_count, escalation_required
workflow.workflow_schedule_approvals	Activation and change approval records.	schedule_id, approver_user_id/group_id, approval_status, decision_reason
workflow.workflow_notifications	In-app notifications and future email/Teams queue.	recipient_user_id, notification_type, title, status
workflow.authorization_decisions	Persisted authorization decisions for sensitive operations.	subject_user_id/agent_id, object_type, action, decision, reason_json
workflow.workflow_schedule_history	Change history snapshot for schedule configuration.	schedule_id, change_type, before_json, after_json

10.2 Entity Relationship Summary

```

registry.workflows 1 -> N workflow.workflow_schedules
workflow.workflow_schedules 1 -> N workflow.workflow_schedule_agent_assignments
registry.ai_agents 1 -> N workflow.workflow_schedule_agent_assignments
registry.ai_models 1 -> N workflow.workflow_schedule_agent_assignments
workflow.workflow_schedules 1 -> N workflow.workflow_runs
workflow.workflow_runs 1 -> N workflow.workflow_run_steps
workflow.workflow_runs 1 -> N workflow.workflow_run_outputs
workflow.workflow_runs 1 -> N workflow.workflow_run_failures
workflow.workflow_schedules 1 -> N workflow.workflow_schedule_approvals
core.users / core.approval_groups -> workflow.workflow_schedule_approvals
workflow.workflow_runs -> audit.audit_events through entity_type/entity_id

```

10.3 Standard Columns for All Phase 2 Tables

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES core.tenants(id),
status VARCHAR(50) NOT NULL DEFAULT 'ACTIVE',
metadata_json JSONB NOT NULL DEFAULT '{}'::jsonb,
created_by UUID NULL REFERENCES core.users(id),
created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_by UUID NULL REFERENCES core.users(id),
updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
version_no INTEGER NOT NULL DEFAULT 1,
is_deleted BOOLEAN NOT NULL DEFAULT FALSE

```

10.4 PostgreSQL DDL Starter Script

```

CREATE TABLE IF NOT EXISTS workflow.workflow_schedules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id),
    workflow_id UUID NOT NULL REFERENCES registry.workflows(id),
    schedule_code VARCHAR(100) NOT NULL,
    schedule_name VARCHAR(255) NOT NULL,
    schedule_description TEXT NULL,
    schedule_type VARCHAR(50) NOT NULL, -- MANUAL, ONE_TIME, INTERVAL, DAILY, WEEKLY, MONTHLY, CRON
    cron_expression VARCHAR(120) NULL,
    timezone VARCHAR(100) NOT NULL DEFAULT 'Asia/Kolkata',
    start_at TIMESTAMPTZ NULL,
    end_at TIMESTAMPTZ NULL,
    next_run_at TIMESTAMPTZ NULL,
    last_run_at TIMESTAMPTZ NULL,
    concurrency_policy VARCHAR(50) NOT NULL DEFAULT 'SKIP_IF_RUNNING',
    max_runtime_seconds INTEGER NOT NULL DEFAULT 1800,
    retry_policy_json JSONB NOT NULL DEFAULT '{"max_retries":1,"retry_delay_seconds":300}'::jsonb,
    owner_user_id UUID NOT NULL REFERENCES core.users(id),
    owner_department_id UUID NULL REFERENCES core.departments(id),
    approval_required BOOLEAN NOT NULL DEFAULT FALSE,
    approval_group_id UUID NULL REFERENCES core.approval_groups(id),

```

```

risk_level VARCHAR(50) NOT NULL DEFAULT 'MEDIUM',
schedule_status VARCHAR(50) NOT NULL DEFAULT 'DRAFT',
metadata_json JSONB NOT NULL DEFAULT '{}'::jsonb,
created_by UUID NULL REFERENCES core.users(id),
created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_by UUID NULL REFERENCES core.users(id),
updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
version_no INTEGER NOT NULL DEFAULT 1,
is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
CONSTRAINT uq_workflow_schedule_code UNIQUE (tenant_id, schedule_code),
CONSTRAINT chk_schedule_window CHECK (end_at IS NULL OR start_at IS NULL OR end_at > start_at)
);

CREATE INDEX IF NOT EXISTS idx_workflow_schedules_tenant ON workflow.workflow_schedules(tenant_id);
CREATE INDEX IF NOT EXISTS idx_workflow_schedules_workflow ON workflow.workflow_schedules(workflow_id);
CREATE INDEX IF NOT EXISTS idx_workflow_schedules_next_run ON workflow.workflow_schedules(next_run_at);
CREATE INDEX IF NOT EXISTS idx_workflow_schedules_status ON workflow.workflow_schedules(schedule_status);
CREATE INDEX IF NOT EXISTS idx_workflow_schedules_owner ON workflow.workflow_schedules(owner_user_id);
CREATE INDEX IF NOT EXISTS idx_workflow_schedules_metadata_gin ON workflow.workflow_schedules USING
GIN(metadata_json);

```

10.5 Additional DDL for Critical Tables

```

CREATE TABLE IF NOT EXISTS workflow.workflow_schedule_agent_assignments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id),
    schedule_id UUID NOT NULL REFERENCES workflow.workflow_schedules(id) ON DELETE CASCADE,
    agent_id UUID NOT NULL REFERENCES registry.ai_agents(id),
    model_id UUID NULL REFERENCES registry.ai_models(id),
    assignment_role VARCHAR(50) NOT NULL DEFAULT 'PRIMARY',
    execution_mode VARCHAR(50) NOT NULL DEFAULT 'RECOMMEND_ONLY',
    confidence_threshold NUMERIC(5,2) NULL,
    allowed_tools_json JSONB NOT NULL DEFAULT '[]'::jsonb,
    allowed_data_sources_json JSONB NOT NULL DEFAULT '[]'::jsonb,
    blocked_operations_json JSONB NOT NULL DEFAULT '[]'::jsonb,
    boundary_rules_json JSONB NOT NULL DEFAULT '{}'::jsonb,
    status VARCHAR(50) NOT NULL DEFAULT 'ACTIVE',
    metadata_json JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_by UUID NULL REFERENCES core.users(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_by UUID NULL REFERENCES core.users(id),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    version_no INTEGER NOT NULL DEFAULT 1,
    is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT uq_schedule_agent_assignment UNIQUE (tenant_id, schedule_id, agent_id, assignment_role),
    CONSTRAINT chk_assignment_confidence CHECK (confidence_threshold IS NULL OR (confidence_threshold >= 0 AND
confidence_threshold <= 100))
);

CREATE TABLE IF NOT EXISTS workflow.workflow_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES core.tenants(id),
    schedule_id UUID NOT NULL REFERENCES workflow.workflow_schedules(id),
    workflow_id UUID NOT NULL REFERENCES registry.workflows(id),
    run_code VARCHAR(120) NOT NULL,
    trigger_type VARCHAR(50) NOT NULL, -- SCHEDULED, MANUAL, EVENT, API
    triggered_by_user_id UUID NULL REFERENCES core.users(id),
    triggered_by_actor_type VARCHAR(50) NOT NULL DEFAULT 'SYSTEM',
    run_status VARCHAR(50) NOT NULL DEFAULT 'QUEUED', -- QUEUED, RUNNING, COMPLETED, FAILED, CANCELLED, SKIPPED
    started_at TIMESTAMPTZ NULL,
    completed_at TIMESTAMPTZ NULL,
    duration_ms BIGINT NULL,
    risk_level VARCHAR(50) NULL,

```

```

summary TEXT NULL,
context_json JSONB NOT NULL DEFAULT '{} '::jsonb,
result_json JSONB NOT NULL DEFAULT '{} '::jsonb,
metadata_json JSONB NOT NULL DEFAULT '{} '::jsonb,
created_by UUID NULL REFERENCES core.users(id),
created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_by UUID NULL REFERENCES core.users(id),
updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
version_no INTEGER NOT NULL DEFAULT 1,
is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
CONSTRAINT uq_workflow_run_code UNIQUE (tenant_id, run_code)
);

CREATE INDEX IF NOT EXISTS idx_workflow_runs_schedule ON workflow.workflow_runs(schedule_id);
CREATE INDEX IF NOT EXISTS idx_workflow_runs_status ON workflow.workflow_runs(run_status);
CREATE INDEX IF NOT EXISTS idx_workflow_runs_created ON workflow.workflow_runs(created_at DESC);
CREATE INDEX IF NOT EXISTS idx_workflow_runs_result_gin ON workflow.workflow_runs USING GIN(result_json);

```

11. API Contract Specification

11.1 API Standards

- All APIs must be under /api/v1/workflow-scheduler or /api/v1/workflow-runs.
- All responses must use standard envelope: success, data, error, request_id, timestamp.
- All write APIs must publish governance events.
- List APIs must support pagination, sorting and filters.
- Every sensitive API must call authorization decision service before execution.

11.2 Endpoint Inventory

Method	Endpoint	Purpose	Permission
GET	/api/v1/workflow-scheduler/schedules	List schedules with filters.	VIEW_WORKFLOW_SCHEDULE
POST	/api/v1/workflow-scheduler/schedules	Create schedule.	CREATE_WORKFLOW_SCHEDULE
GET	/api/v1/workflow-scheduler/schedules/{id}	Get schedule detail.	VIEW_WORKFLOW_SCHEDULE
PUT	/api/v1/workflow-scheduler/schedules/{id}	Update draft/paused schedule.	UPDATE_WORKFLOW_SCHEDULE
POST	/api/v1/workflow-scheduler/schedules/{id}/submit	Submit for approval.	SUBMIT_WORKFLOW_SCHEDULE
POST	/api/v1/workflow-scheduler/schedules/{id}/activate	Activate approved schedule.	ACTIVATE_WORKFLOW_SCHEDULE
POST	/api/v1/workflow-scheduler/schedules/{id}/pause	Pause active schedule.	PAUSE_WORKFLOW_SCHEDULE
POST	/api/v1/workflow-scheduler/schedules/{id}/resume	Resume paused schedule.	RESUME_WORKFLOW_SCHEDULE
POST	/api/v1/workflow-scheduler/schedules/{id}/retire	Retire schedule.	RETIRE_WORKFLOW_SCHEDULE
POST	/api/v1/workflow-scheduler/schedules/{id}/run-now	Trigger manual run.	RUN_WORKFLOW_SCHEDULE
GET	/api/v1/workflow-runs	List workflow runs.	VIEW_WORKFLOW_RUN
GET	/api/v1/workflow-runs/{run_id}	Get run detail.	VIEW_WORKFLOW_RUN
GET	/api/v1/workflow-runs/{run_id}/steps	Get run steps.	VIEW_WORKFLOW_RUN
GET	/api/v1/workflow-runs/{run_id}/outputs	Get run outputs.	VIEW_WORKFLOW_RUN_OUTPUT
POST	/api/v1/workflow-runs/{run_id}/cancel	Cancel running run.	CANCEL_WORKFLOW_RUN
POST	/api/v1/authorization/evaluate	Evaluate authorization decision.	EVALUATE_AUTHORIZATION

11.3 Create Schedule Request Example

```

{
  "workflow_id": "uuid-of-registered-workflow",
  "schedule_code": "DAILY_MODEL_RISK_REVIEW",
  "schedule_name": "Daily AI Model Risk Review",
  "schedule_type": "DAILY",

```

```
"timezone": "Asia/Kolkata",
"start_at": "2026-06-17T09:00:00+05:30",
"concurrency_policy": "SKIP_IF_RUNNING",
"max_runtime_seconds": 1800,
"owner_user_id": "uuid-owner",
"approval_required": true,
"approval_group_id": "uuid-ai-governance-board",
"risk_level": "HIGH",
"agent_assignments": [
  {
    "agent_id": "uuid-agent",
    "model_id": "uuid-model",
    "execution_mode": "RECOMMEND_ONLY",
    "confidence_threshold": 80,
    "allowed_tools": ["REGISTRY_READ_API", "AUDIT_READ_API"],
    "blocked_operations": ["UPDATE_POLICY", "EXECUTE_EXTERNAL_ACTION"],
    "boundary_rules": {
      "max_records": 500,
      "allow_write_tools": false,
      "requires_human_approval_for_high_risk": true
    }
  }
]
```

11.4 Standard Response Example

```
{
  "success": true,
  "data": {
    "schedule_id": "uuid",
    "schedule_status": "PENDING_APPROVAL",
    "approval_required": true,
    "approval_reason": "High risk schedule with AI agent execution boundary requires AI Governance Board approval"
  },
  "error": null,
  "request_id": "REQ-20260616-0001",
  "timestamp": "2026-06-16T10:30:00Z"
}
```

12. Scheduling and Execution Engine Design

12.1 Scheduler Engine Options

Option	When to Use	Recommendation
APScheduler inside FastAPI worker	Simple initial implementation, local deployment, limited scale.	Good for Phase 2 MVP.
Celery Beat + Celery Workers	Distributed execution, retries, production-grade async tasks.	Recommended for scalable release.
AWS EventBridge + Lambda/ECS	Cloud-native scheduling with AWS deployment.	Recommended if deployment is AWS-first.
Database polling worker	Simple and transparent; worker checks due schedules every minute.	Acceptable first version if built carefully with locking.

Recommended initial implementation: database-backed scheduler worker with row-level locking, then upgrade to Celery Beat or AWS EventBridge when deployment architecture is finalized.

12.2 Due Schedule Query Pattern

```
SELECT *
FROM workflow.workflow_schedules
WHERE schedule_status = 'ACTIVE'
```

```

AND is_deleted = false
AND next_run_at <= CURRENT_TIMESTAMP
ORDER BY next_run_at ASC
FOR UPDATE SKIP LOCKED
LIMIT 25;

```

12.3 Run State Machine

```

QUEUED -> RUNNING -> COMPLETED
QUEUED -> RUNNING -> FAILED
QUEUED -> SKIPPED
RUNNING -> CANCELLED
FAILED -> RETRY_QUEUED -> RUNNING
COMPLETED is terminal.
CANCELLED is terminal.
SKIPPED is terminal.

```

12.4 Schedule Status Lifecycle

```

DRAFT -> PENDING_APPROVAL -> ACTIVE -> PAUSED -> ACTIVE -> RETIRED
DRAFT -> ACTIVE only if approval_required = false and authorization allows activation.
ACTIVE -> FAILED only for systemic schedule configuration failure; individual run failures do not automatically fail the schedule.
RETIRED is terminal and historical runs must remain available.

```

13. RBAC + ABAC Authorization Design

Phase 2 must reuse Domain 1 authorization philosophy: Authorization Decision = RBAC Permission + ABAC Context + Relationship Check. This is critical because schedule activation depends on user authority, workflow ownership, risk level, department, execution mode and tool sensitivity.

Action	RBAC Permission	ABAC / Relationship Conditions
Create schedule	CREATE_WORKFLOW_SCHEDULE	User belongs to owner department or has governance admin scope.
Assign agent	ASSIGN_AI_AGENT_TO_WORKFLOW	User can manage selected agent/model or is governance admin.
Activate schedule	ACTIVATE_WORKFLOW_SCHEDULE	If high risk, user must be approval group member or risk manager; delegation allowed if active.
Run now	RUN_WORKFLOW_SCHEDULE	Schedule must be active; user can run schedule within department/BU scope.
View outputs	VIEW_WORKFLOW_RUN_OUTPUT	User has audit/review permission and output sensitivity is within access scope.
Cancel run	CANCEL_WORKFLOW_RUN	User is schedule owner, governance admin or escalation owner.
Override failure	OVERRIDE_WORKFLOW_FAILURE	Must capture reason; only explicit override roles allowed.

13.1 Authorization Decision Pseudocode

```

def evaluate_schedule_action(subject, action, schedule, context):
    if not rbac_allows(subject.roles, action):
        return Deny("RBAC permission missing")

    if schedule.risk_level in ["HIGH", "CRITICAL"] and action in ["ACTIVATE", "RUN_NOW"]:
        if not is_approval_group_member(subject.user_id, schedule.approval_group_id):
            if not has_active_delegation(schedule.assigned_approver, subject.user_id, schedule):
                return Deny("High risk schedule requires approval group membership or valid delegation")

    if action == "ASSIGN_AGENT":
        if requested_execution_mode_exceeds_agent_registry_boundary(schedule.agent_id):
            return Deny("Requested execution mode exceeds registered agent boundary")

    if contains_write_tool(schedule.allowed_tools) and not schedule.approval_required:
        return Deny("Write-capable tool requires approval")

```

```
return Allow("Authorized")
```

14. Governance Event and Audit Design

Every material action in Phase 2 must publish an immutable governance event into audit.audit_events. Events make the lifecycle reconstructable and support executive trust, compliance review and debugging.

Event	When Generated	Actor
WORKFLOW_SCHEDULE_CREATED	Schedule created in DRAFT.	Schedule creator
WORKFLOW_SCHEDULE_UPDATED	Schedule configuration changed.	Schedule owner/admin
WORKFLOW_SCHEDULE_SUBMITTED	Schedule submitted for approval.	Schedule owner
WORKFLOW_SCHEDULE_APPROVED	Schedule approved for activation.	Approver or approval group member
WORKFLOW_SCHEDULE_ACTIVATED	Schedule made active.	Authorized user/system
WORKFLOW_SCHEDULE_PAUSED	Schedule paused.	Owner/admin
WORKFLOW_SCHEDULE_RETIRED	Schedule retired.	Owner/admin
WORKFLOW_RUN_QUEUED	Run created and queued.	System or manual trigger user
WORKFLOW_RUN_STARTED	Run execution started.	System
AGENT_BOUNDARY_CHECK_PASSED	Agent boundary check succeeded.	System
AGENT_BOUNDARY_CHECK_FAILED	Agent boundary check failed.	System
AGENT_EXECUTION_STARTED	Agent invocation started.	Machine actor/system
AGENT_EXECUTION_COMPLETED	Agent invocation completed.	Machine actor/system
WORKFLOW_OUTPUT_GENERATED	Findings/recommendations/evidence generated.	Machine actor/system
WORKFLOW_RUN_COMPLETED	Run completed successfully.	System
WORKFLOW_RUN_FAILED	Run failed.	System
WORKFLOW_RUN_ESCALATED	Failure or high-risk finding escalated.	System
AUTHORIZATION_DENIED	Unauthorized attempt blocked.	System

14.1 Audit Event Payload Example

```
{
  "event_code": "WORKFLOW_RUN_COMPLETED",
  "event_category": "WORKFLOW_EXECUTION",
  "entity_type": "WORKFLOW_RUN",
  "entity_id": "run_uuid",
  "actor_type": "SYSTEM",
  "action_type": "COMPLETE",
  "event_summary": "Daily AI Model Risk Review completed with 2 medium findings and 1 high finding.",
  "event_payload": {
    "schedule_id": "schedule_uuid",
    "workflow_id": "workflow_uuid",
    "agent_id": "agent_uuid",
    "duration_ms": 42000,
    "risk_level": "HIGH",
    "outputs": {
      "findings_count": 3,
      "recommendations_count": 2
    }
  }
}
```

15. Notifications, Escalations and SLA Handling

Trigger	Notification Recipient	Action
Schedule requires approval	Approval group members, owner	Create approval notification and in-app task.
Schedule activation approved/rejected	Schedule owner	Notify decision and reason.
Run failed	Schedule owner, escalation owner	Notify with failure type and retry status.

Run SLA breached	Escalation owner, governance admin	Escalate and create audit event.
High-risk output generated	Risk manager, compliance officer, owner	Create review task and notification.
Authorization denied	Actor, owner, security/governance admin depending severity	Log event and notify for sensitive actions.
Schedule paused automatically	Owner, admin	Notify with reason and required action.

16. Error Handling and Failure Recovery

Failure Type	Example	Required Handling
VALIDATION_ERROR	Invalid cron expression, missing owner, inactive agent.	Return 400 with field-level errors; do not create/activate.
AUTHORIZATION_DENIED	User tries to activate outside scope.	Return 403; publish AUTHORIZATION_DENIED.
REGISTRY_REFERENCE_ERROR	Agent or workflow no longer active.	Block execution; fail run with clear reason.
BOUNDARY_VIOLATION	Agent attempts blocked tool operation.	Stop run; log AGENT_BOUNDARY_CHECK_FAILED.
MODEL_INVOCATION_ERROR	LLM/API timeout or failure.	Retry if configured; otherwise fail run.
OUTPUT_PARSE_ERROR	Agent output not valid JSON.	Store raw output, mark parse failure, escalate if required.
DATABASE_ERROR	Insert/update failure.	Rollback transaction, log technical error, return 500.
CONCURRENCY_CONFLICT	Previous run still running.	Skip or queue based on concurrency policy.
SLA_BREACH	Run exceeds max_runtime_seconds.	Cancel/fail run and notify escalation owner.

17. Security, Privacy and Compliance Controls

- All schedule and run APIs must require authenticated user context or approved service account context.
- No API should trust frontend-provided user role or department. Authorization must resolve from backend identity tables.
- Agent execution must never dynamically access a tool unless that tool is explicitly registered and allowed in assignment boundary.
- Sensitive run outputs must be masked or visibility-controlled based on data sensitivity and user scope.
- Secrets must not be stored in workflow schedule JSON. Use secret manager or environment-based configuration.
- Run output raw payloads should be classified and retention-controlled.
- Authorization denied events should be generated for sensitive failed attempts.
- Schedule changes must preserve history; do not overwrite critical governance evidence without change log.

18. Integration Design

Integration	Phase 2 Use	Implementation Note
Phase 1 Registry	Read workflows, agents, models, tools, data sources.	Use foreign keys and service APIs.
Audit Events	Publish schedule and run lifecycle events.	Use audit.audit_events and event service.
SignalIQ	Optional source of governance workflows or signals.	Start with read-only demo integration.
Notification Channels	Email/Teams/Slack future integration.	Start with workflow_notifications table.
Policy Engine	Future Phase 3 policy checks.	Create lightweight hook interface now.
Approval Module	Schedule activation approvals.	Implement minimal schedule approval in Phase 2; extend later.
Agent Runtime/LLM Gateway	Invoke configured model/agent.	Use adapter pattern to avoid vendor lock-in.

19. Development Work Breakdown Structure

WBS	Workstream	Activity	Owner Role
2.1	Data Model	Create workflow schema tables, indexes, triggers, seed data.	Backend/Data Engineer

2.2	Backend Models	Create SQLAlchemy models and Pydantic schemas.	Backend Developer
2.3	Schedule APIs	Implement schedule CRUD, status lifecycle and validation.	Backend Developer
2.4	Agent Assignment APIs	Implement assignment, boundary validation and registry checks.	Backend Developer
2.5	Run Engine	Implement run creation, state transitions, run steps and outputs.	Backend Developer
2.6	Scheduler Worker	Implement due schedule polling and execution trigger.	Backend Developer
2.7	Authorization Service	Implement RBAC + ABAC checks for Phase 2 actions.	Backend Developer
2.8	Audit Events	Publish events for all lifecycle actions.	Backend Developer
2.9	Frontend Scheduler Dashboard	List/filter schedule records and actions.	Frontend Developer
2.10	Create Schedule Wizard	Multi-step wizard with validation and summary.	Frontend Developer
2.11	Run History and Detail	Run list, run detail, timeline, output viewer.	Frontend Developer
2.12	Approval Queue	Approve/reject schedule activation.	Frontend Developer
2.13	QA Tests	API, UI, integration, authorization, negative tests.	QA Engineer
2.14	DevOps	Migration scripts, env configs, worker process setup.	DevOps/Backend
2.15	Documentation	Developer readme, API examples and deployment notes.	Tech Lead/PM

20. Testing and Acceptance Criteria

20.1 Test Scenarios

Test ID	Scenario	Expected Result
TC-001	Create valid daily schedule with recommend-only agent.	Schedule created in DRAFT or ACTIVE based on approval logic.
TC-002	Create schedule with inactive agent.	API rejects with clear validation error.
TC-003	Activate high-risk schedule as unauthorized user.	403 returned; AUTHORIZATION DENIED logged.
TC-004	Activate high-risk schedule as approval group member.	Schedule becomes ACTIVE; event logged.
TC-005	Manual run active schedule.	Run created, steps executed, status COMPLETED.
TC-006	Run schedule with blocked tool operation.	Run fails; boundary violation event logged.
TC-007	Scheduler worker detects due schedule.	Run created automatically and next_run_at updated.
TC-008	Concurrent run when policy is SKIP_IF_RUNNING.	Second run skipped; event logged.
TC-009	Run output contains high-risk finding.	Notification/escalation created.
TC-010	Auditor opens run detail.	Read-only view with audit timeline visible.
TC-011	Business user outside department tries to view output.	Access denied or output hidden based on ABAC.
TC-012	Pause schedule.	No automatic runs occur while paused.
TC-013	Retire schedule.	Schedule cannot be resumed; historical runs remain.
TC-014	Invalid cron expression.	Validation error returned before save.
TC-015	Failed run retry configured.	Run retries according to retry policy and logs retry events.

20.2 Definition of Done

- All Phase 2 tables and migrations are checked in and tested on clean database.
- All APIs are documented in Swagger/OpenAPI with sample payloads.
- Frontend screens are integrated with backend APIs and not static mocks.
- Scheduler worker runs due schedules and records run history.
- RBAC + ABAC checks are applied on sensitive actions.
- Governance events are generated for schedule and run lifecycle.
- At least 15 functional test cases and 10 negative authorization tests pass.
- Demo seed data includes at least one model, one agent, one workflow and one daily schedule.
- Run detail screen shows steps, output, failures and audit timeline.
- PM/Tech Lead sign-off is captured against acceptance criteria.

21. Deployment and Migration Guide

14. Backup current Phase 0 and Phase 1 database before applying Phase 2 migrations.
15. Apply workflow schema migration scripts in lower environment first.

16. Run seed data for Phase 2 permissions and lookup values.
17. Deploy backend with new API routes disabled behind feature flag if needed.
18. Start scheduler worker separately from API process.
19. Validate due schedule polling against sample data.
20. Deploy frontend routes and menu entries.
21. Run smoke tests: create schedule, activate, run now, view run detail.
22. Enable automatic scheduler after manual test success.
23. Monitor audit events and worker logs for first execution cycle.

22. Seed Data and Sample Scenarios

22.1 Recommended Demo Scenario

Daily AI Model Risk Review

Field	Value
Workflow	AI Recommendation Review Workflow
Agent	Governance Review Agent
Model	Risk Classification Model or General Purpose LLM
Schedule	Daily at 09:00 IST
Execution Mode	RECOMMEND_ONLY
Allowed Tools	Registry Read API, Audit Read API
Blocked Operations	Update policy, execute external action, send email without approval
Owner	AI Governance Admin
Approval Group	AI Governance Board
Output	Findings, risk score, recommended actions and evidence summary

22.2 Seed Permissions

CREATE_WORKFLOW_SCHEDULE
 VIEW_WORKFLOW_SCHEDULE
 UPDATE_WORKFLOW_SCHEDULE
 SUBMIT_WORKFLOW_SCHEDULE
 ACTIVATE_WORKFLOW_SCHEDULE
 PAUSE_WORKFLOW_SCHEDULE
 RESUME_WORKFLOW_SCHEDULE
 RETIRE_WORKFLOW_SCHEDULE
 RUN_WORKFLOW_SCHEDULE
 VIEW_WORKFLOW_RUN
 VIEW_WORKFLOW_RUN_OUTPUT
 CANCEL_WORKFLOW_RUN
 ASSIGN_AI_AGENT_TO_WORKFLOW
 EVALUATE_AUTHORIZATION

23. Implementation Risks and Controls

Risk	Impact	Control
Scheduler built as simple cron without governance context	Weak auditability and uncontrolled execution.	Use schedule/run/event model from this spec.
Agent can access tools not explicitly allowed	Potential unauthorized action.	Boundary check before every invocation.
Overlapping runs overload system	Duplicate outputs and inconsistent results.	Concurrency policy and run locks.
No approval before high-risk schedule activation	Governance gap.	Mandatory approval_required logic based on risk/execution/tool sensitivity.
Frontend bypasses validation	Bad configuration saved.	Backend validations are authoritative.
Run outputs contain sensitive data	Unauthorized exposure.	Output classification and ABAC visibility controls.
Audit events are incomplete	Weak traceability.	Event checklist and test cases.
Scheduler worker failure stops all schedules	Operational issue.	Health checks, monitoring and failure

		notifications.
Physical model lacks indexes	Poor performance as runs grow.	Create indexes on status, next_run_at, schedule_id, created_at and JSONB fields where required.

24. Appendices: Payloads and Developer Checklists

24.1 Run Output Example

```
{
  "output_type": "GOVERNANCE_FINDINGS",
  "severity": "HIGH",
  "risk_score": 86,
  "findings": [
    {
      "finding_code": "MODEL_OWNER_MISSING",
      "entity_type": "AI_MODEL",
      "entity_code": "LLM_GENERAL_001",
      "message": "AI model does not have an active accountable owner mapping."
    },
    {
      "finding_code": "AGENT_TOOL_WRITE_ACCESS",
      "entity_type": "AI_AGENT",
      "entity_code": "GOV_REVIEW_AGENT_001",
      "message": "Agent has a write-capable tool configured but schedule approval is not complete."
    }
  ],
  "recommendations": [
    {
      "recommendation_type": "GOVERNANCE_ACTION",
      "priority": "HIGH",
      "recommended_action": "Assign AI Asset Owner and route to AI Governance Board."
    }
  ],
  "evidence": {
    "source_tables": ["registry.ai_models", "registry.ai_agent_tool_mapping"],
    "run_id": "uuid"
  }
}
```

24.2 Backend Developer Checklist

- Migrations created and reversible.
- SQLAlchemy models match DDL.
- Pydantic schemas validate required fields and enum values.
- All write APIs call authorization service.
- All lifecycle changes publish audit event.
- Scheduler worker uses row locking or equivalent concurrency control.
- Run state machine prevents illegal transitions.
- Errors use standard error envelope.
- API tests cover success and failure cases.

24.3 Frontend Developer Checklist

- All forms use backend-driven dropdowns from registry APIs.
- Wizard validates each step before next step.
- High-risk/approval banners clearly explain why approval is required.
- Run timeline is readable and includes failed step details.

- Buttons are hidden or disabled based on permissions and backend decision.
- Validation errors are displayed near fields.
- Loading, empty and error states are implemented.
- Audit timeline is visible in schedule detail and run detail.

24.4 QA Checklist

- Create/activate/run/pause/resume/retire lifecycle tested.
- RBAC negative tests completed.
- ABAC department/risk/sensitivity tests completed.
- Delegation scenario tested if delegation APIs are available.
- Boundary violation tested.
- Run failure and retry tested.
- UI run history matches database records.
- Audit events exist for all critical actions.
- No schedule can run if agent or workflow is inactive.
- All test evidence captured for handover.

24.5 Reference Sources

Reference	Relevance
GuardianIQ Domain 1 Foundation Document	Identity, Access & Accountability foundation used as the primary input for roles, RBAC/ABAC, machine actor, delegation and audit requirements.
NIST AI Risk Management Framework	Used for AI risk management and governance alignment.
NIST SP 800-162 ABAC	Used for attribute-based access control decision pattern.
ISO/IEC 42001	Used for AI management system alignment.
EU AI Act direction	Used for human oversight, monitoring and logging alignment for high-risk AI systems.

Final Recommendation

Do not implement Phase 2 as only a UI scheduler. Implement it as a governed execution layer. The scheduler must be tightly connected to identity, registry, authorization, audit and execution boundaries. This approach ensures GuardianIQ remains an enterprise AI governance platform rather than becoming a generic automation tool.